Singly Linked List – Insert At A Position – 2 And Time Complexity.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}

void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));

    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
```

```
        return;
    }

    newNode->data = data;

    if (position == 1)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }

    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 1)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 1;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}

void insertAtPosition2(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (position == 0)
    {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }

    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        int i = 0;
```

```
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {

        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;
        cout << "3. Insert at a position,indexing node from 1 to N" << endl;
        cout << "4. Insert at a position,indexing node from 0 to N-1" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
```

```
        int data;
        cout << "Enter the data: ";
        cin >> data;
        insertAtBeginning(&head, data);
        break;
    }
  case 2:
  {
        int data;
        cout << "Enter the data: ";
        cin >> data;
        insertAtEnd(&head, data);
        break;
  }
 case 3:
    {
        int data,position;
        cout << "Enter the data: ";
        cin >> data;
        cout << "Enter the position: ";
        cin >> position;
        insertAtPosition(&head, data, position);
        break;
       }

    case 4:
    {
        cout << "Insert at a position,indexing node from 0 to N-1:" << endl;
        int data, position;
        cout << "Enter the data: ";
        cin >> data;
        cout << "Enter the position: ";
        cin >> position;
        insertAtPosition2(&head, data, position);

        break;
    }
```

```
        default:
         {
             cout << "Invalid choice." << endl;
             break;
         }
         }
    }
  return 0;
}
```

Previous Insertion was from 1 *to* $N+1$ *, here insertion will take place* 0 *to* N *, if user count nodes of linked list from* 0 *to* N*. The process is same as discussed earlier, given below*:

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically* 8 *bytes on most modern* $64-bit$ *systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

→ *data(an integer) is typically* 4 *bytes.*

→ *next(a pointer to another Node) is typically* 8 *bytes.*

→ *Therefore , the size of the Node structure is typically* 12 *bytes(* 4 *bytes for data* $+$ 8 *bytes for next) .*

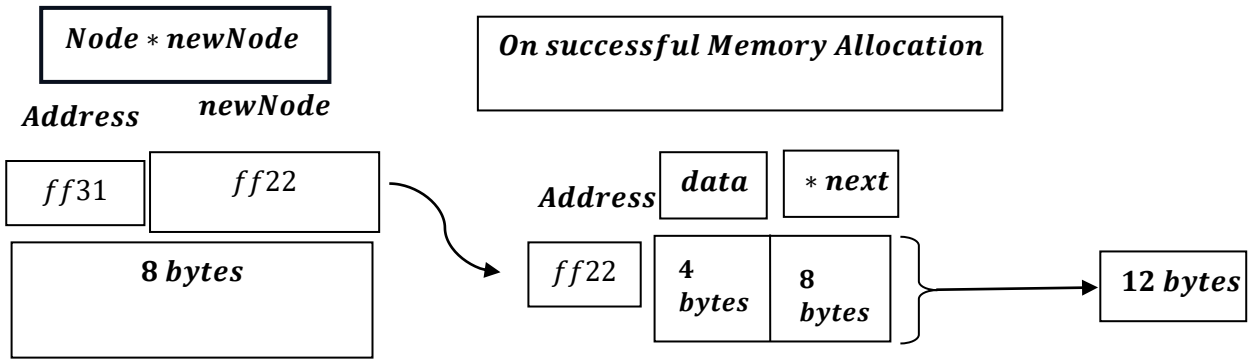
```
    if (newNode == nullptr)
```

```
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
```

What does it mean?

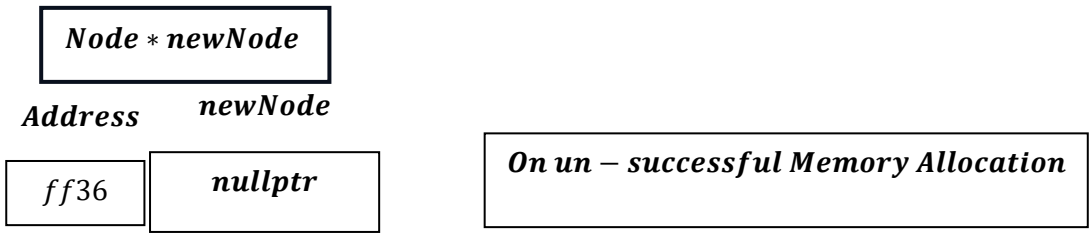
→ *newNode will try to allocate memory for data and next pointer variable i.e. typically 8 bytes, though for variable data and structure Node's pointer variable next in total 12 bytes, but as discussed earlier, the size of a pointer (newNode) is typically 8 bytes on most modern 64 – bit systems, regardless of the size of the structure it points to.*

Say,



But on Un – Successful memory allocation:

→ *if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer.*



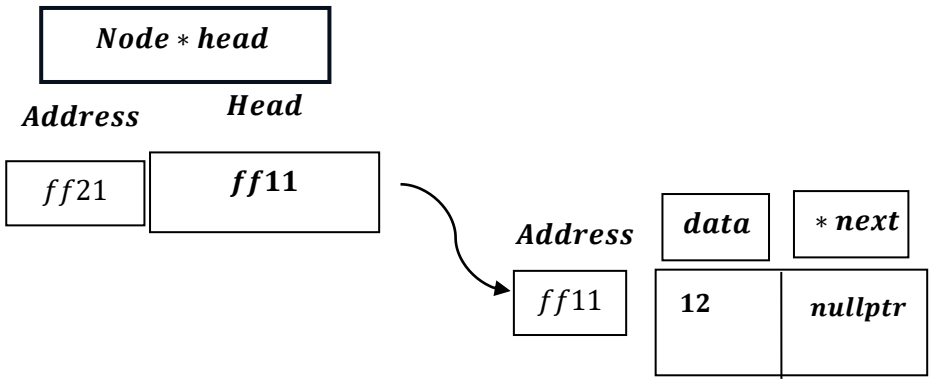
$Now, if\ the\ value\ of\ the\ pointer\ variable\ newNode\ is\ nullptr, then$:

$One\ may\ output$: `Overflow $or\ Memory\ Allocation\ Failed\ as\ a\ message.$`

$Now, we\ move\ from\ Node\ 0\ to\ Node\ N-1$.

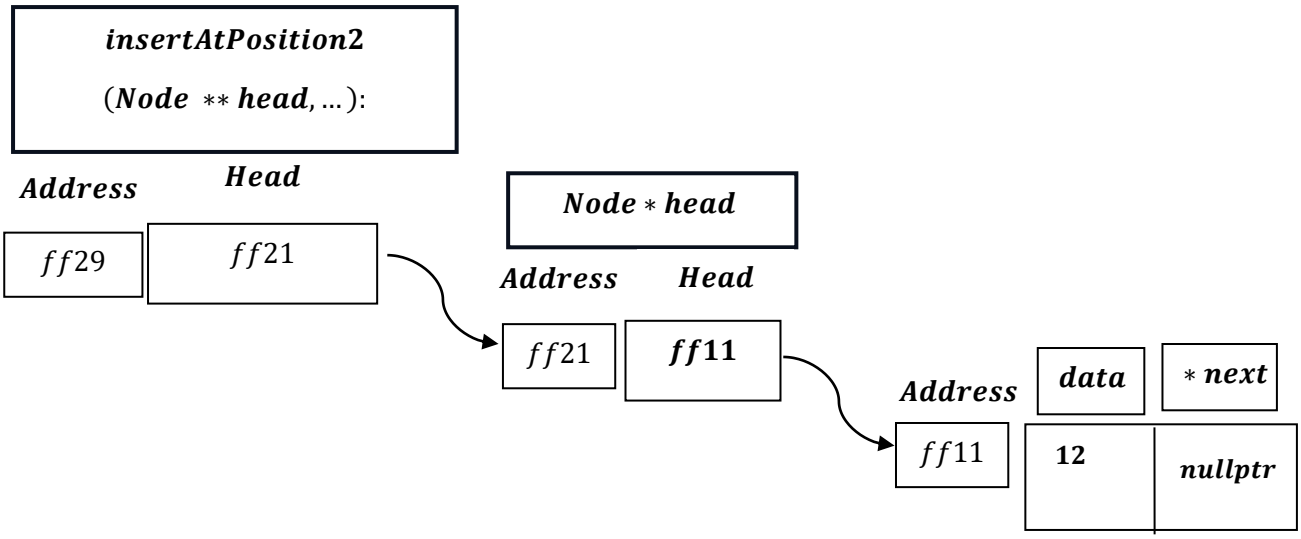
```
case 4:
    {
        cout << "Insert at a position,indexing node from 0 to N-1:" << endl;
        int data, position;
        cout << "Enter the data: ";
        cin >> data;
        cout << "Enter the position: ";
        cin >> position;
        insertAtPosition2(&head, data, position);

        break;
    }
```

$Say, data\ is\ 20$, $and\ position\ is\ 1\ hence\ we\ pass\ through\ function\ call$:



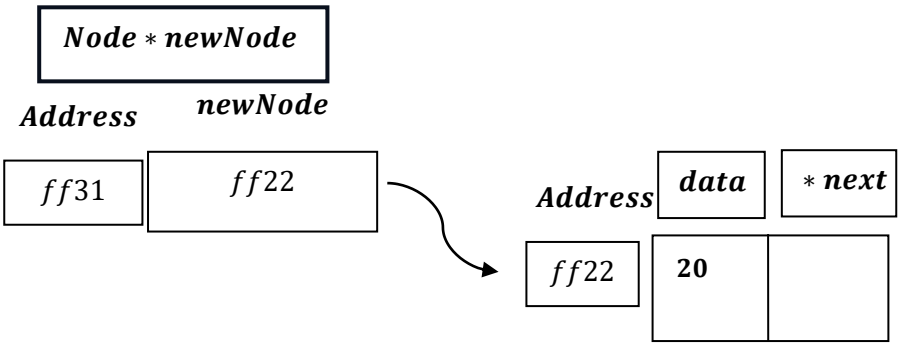
$insertAtPosition2(ff21, 20, 1)$;

```
void insertAtPosition2(Node **head, int data, int position) {…}
```




```
newNode->data = data;
```

The newNode pointer of Node structure points to address of data , assigns the value of data i.e. 20 here.

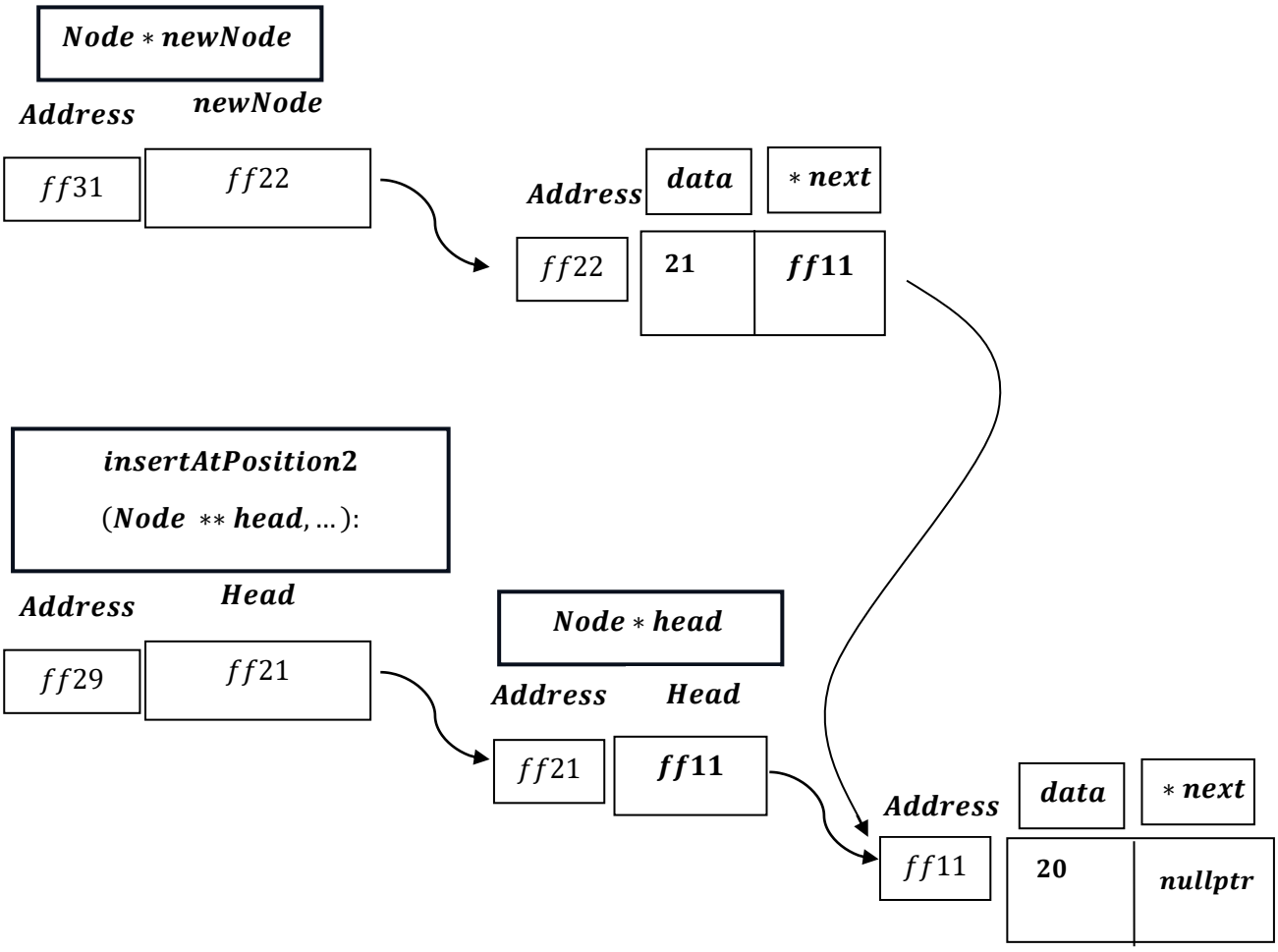



```
if (position == 0)
    {
     newNode->next = *head;
     *head = newNode;
     return;
 }
```

*For newNode → next = * head; i.e. next pointer assigned to the value of head pointer of Node structure. Here value is: ff11.*

```
        newNode->next = *head;
```

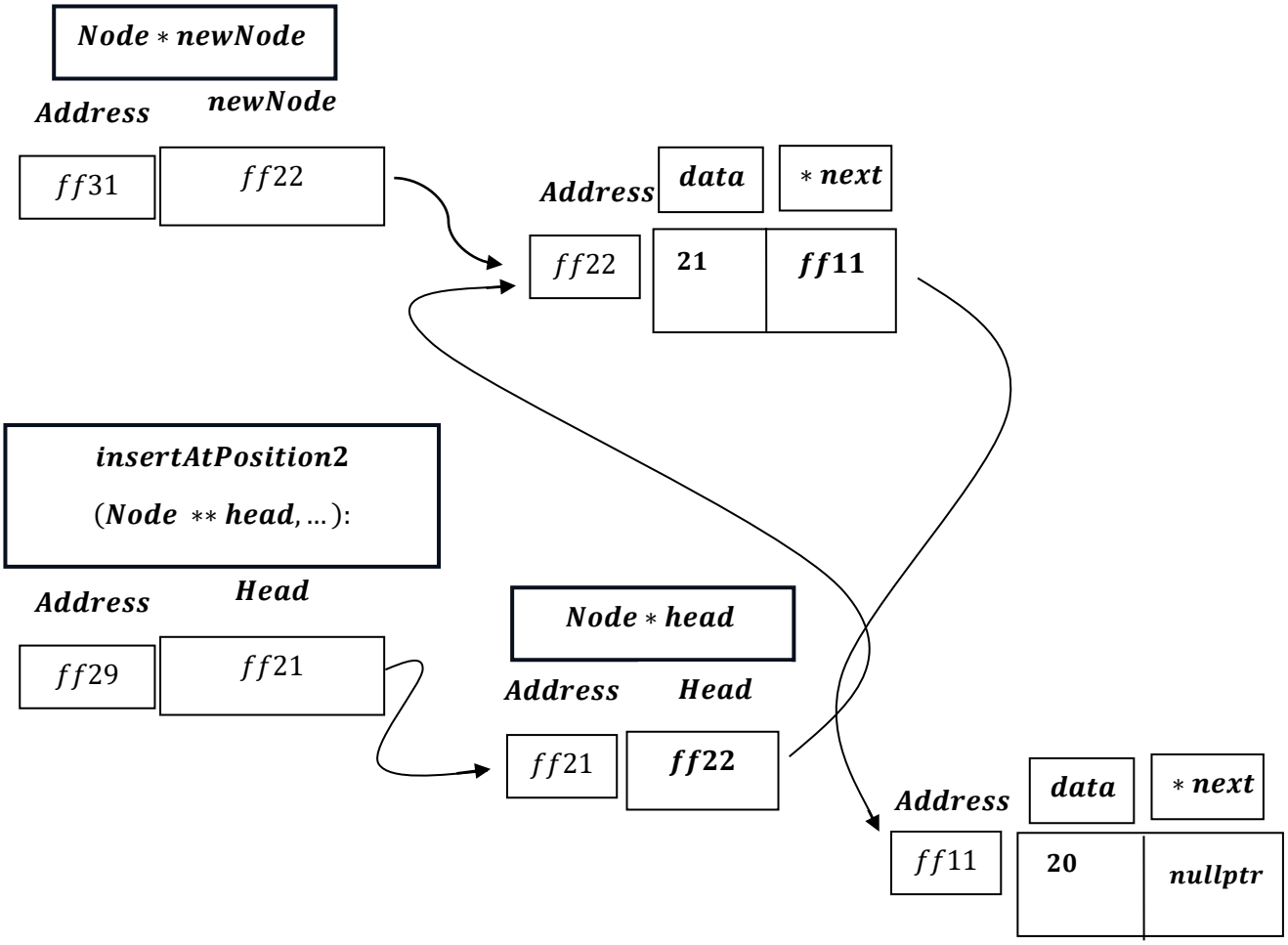
Why ff11 ? newNode → next = ∗ head , here head used is of local pointer to pointer i.e. Node ∗∗ head, and as ∗ head is used, it will first derefer from Node ∗∗ head and refer us to Node ∗ head, which will cause next pointer to store value contained in Node ∗ head i.e. ff11 , address of Head node to which Node ∗ head pointer points to.



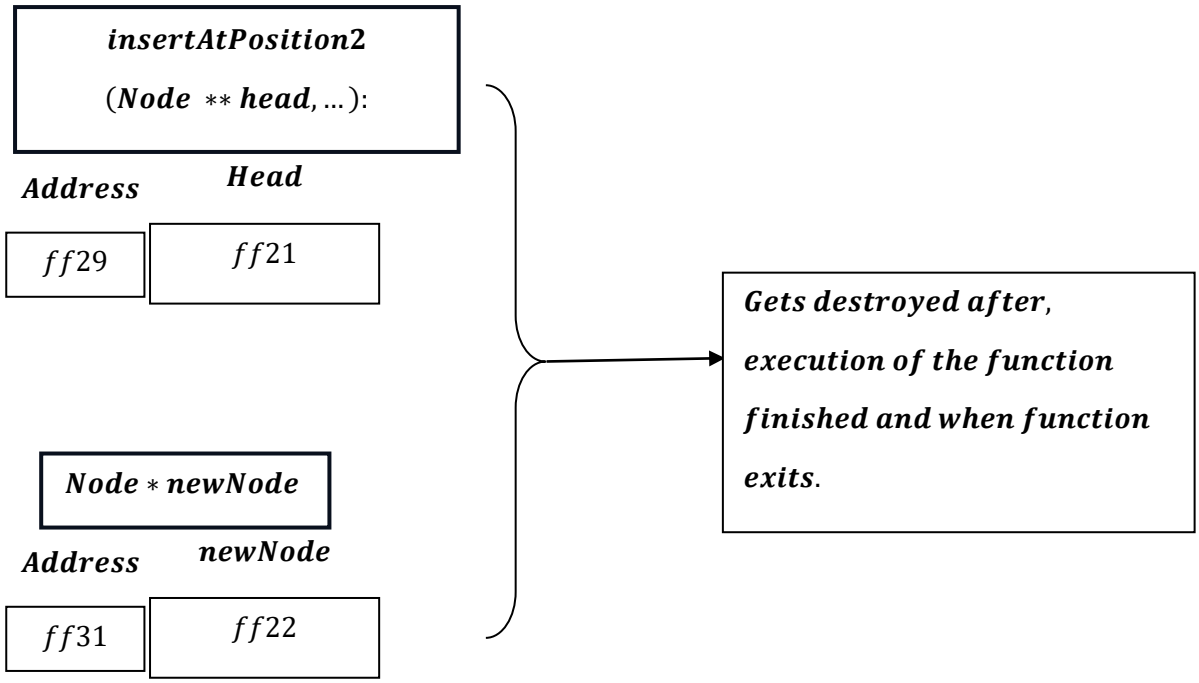

```
        *head = newNode;
```

What does it mean? It means , here head variable used is of Node ∗∗ head , temporary pointer to pointer variable in function, As we use ∗ head , it dereference from Node ∗∗ head and refer us to Node ∗ head, global pointer to which Node ∗∗ head points to. And the line of code ` ∗ head = newNode` change the value of global pointer Node ∗ head , due to dereferencing of

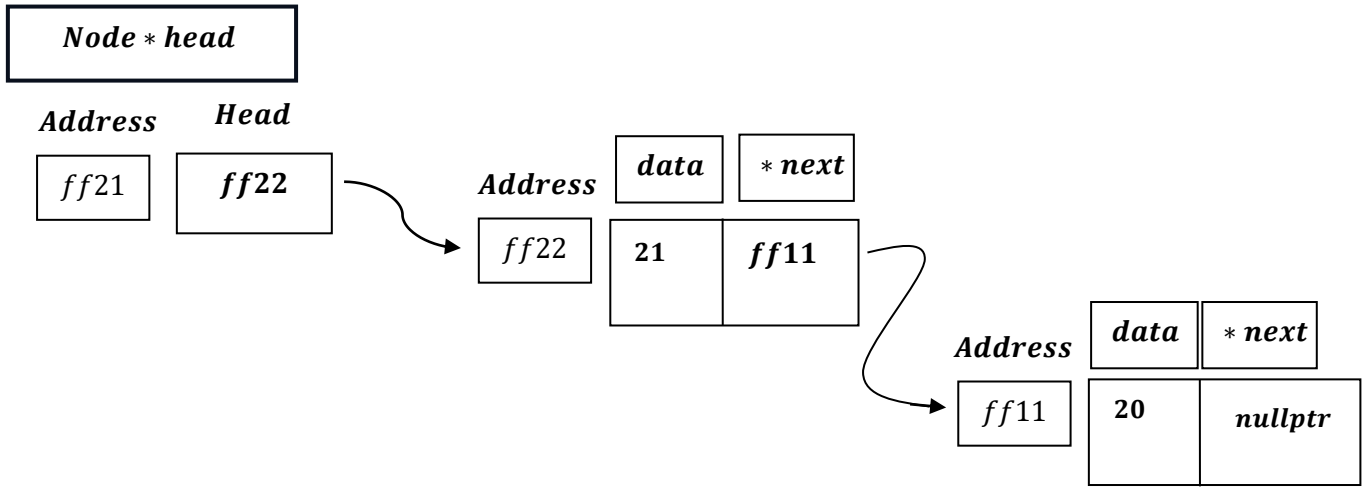
$Node ** head$ to $Node * head$ and $Node * head$ gets assigned to address of $newNode$ pointer variable, which stored as a value in $Node * head[here, ff11]$. And the value i.e. the address of $Node * head[here, ff21]$ stored in $Node ** head$ will remain unchanged and unaltered.



Now, as $Node * newNode$ and $Node ** head$ are temporary variable, therefore gets destroyed after function execution get finished.

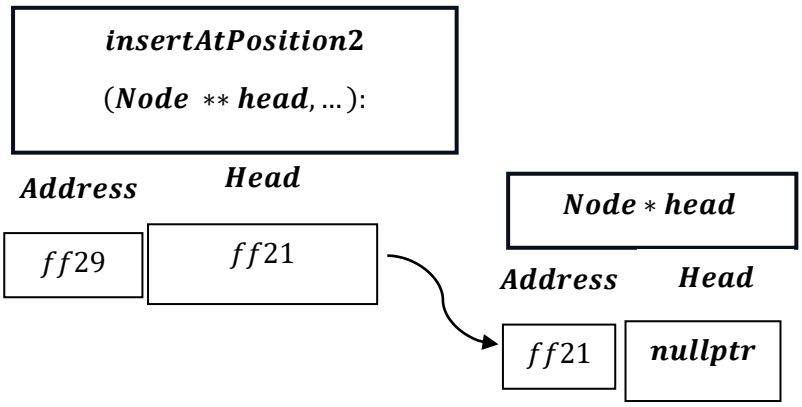


And we are left with:



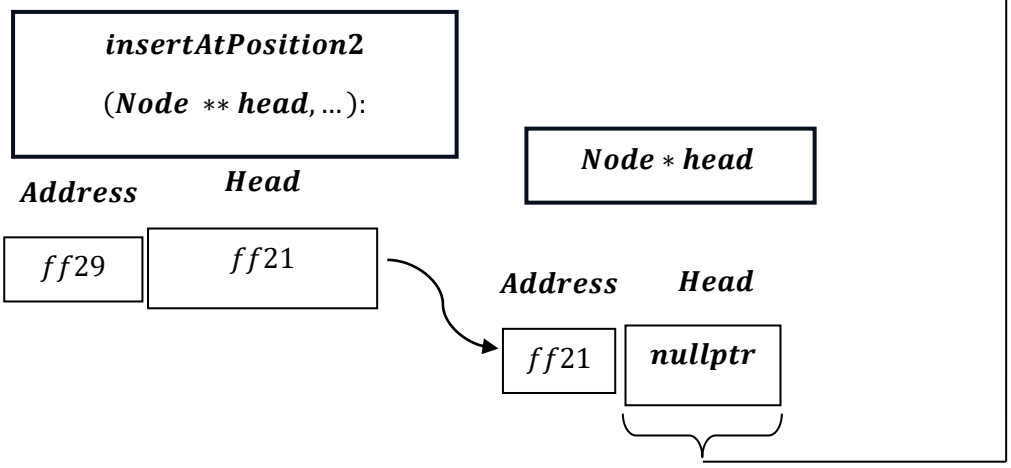
Now, we came up with some error handling scenarios:

```
    if (*head == nullptr)
    {
        cout << "Error: List is empty." << endl;
        return;
    }
```



if(head == nullptr) if it is true*

If true then Why true ?
*Ans: Though * head which is of Node ** head*
temporary pointer to pointer variable of the function
*, as * head is used , it dereference from Node ** head and*
*refer to Node * head global pointer to which Node ** head*
is pointing to and then its value is checked is
whether equal to nullptr or not ? This occurs during execution
and compilation of the insertAtPosition2 function.



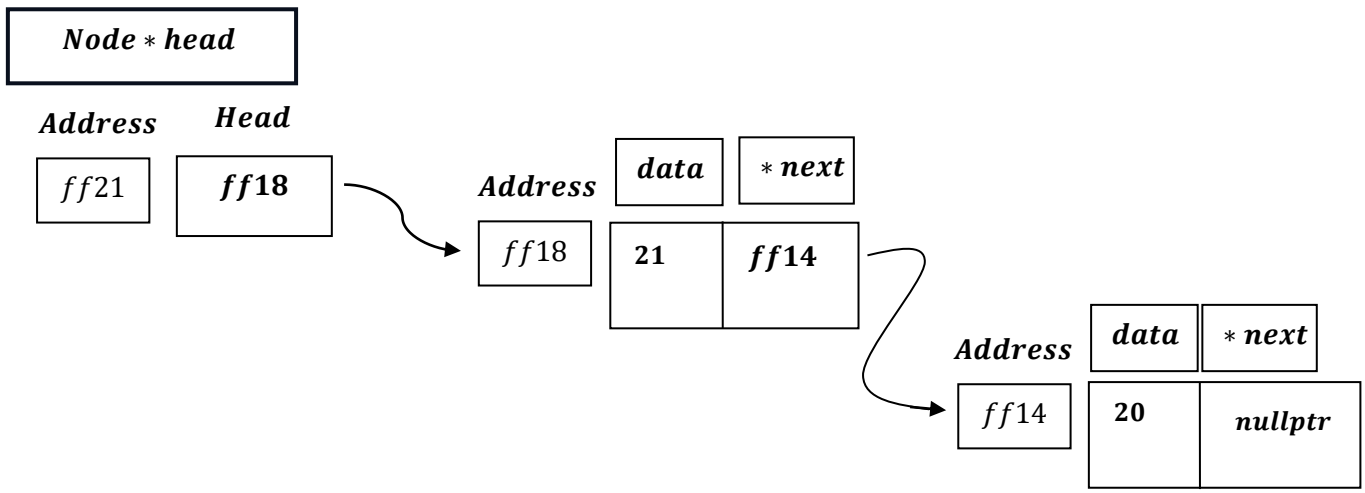
Then , it prints a message : `Error: List is empty.`

```
    if (position < 0)
    {
        cout << "Error: Invalid position." << endl;
        return;
    }
```

If position is less than 0 , then it returns message : ` Error : Invalid position.`

Now , if position is greater than 1 and head doesnot contain nullptr i.e.

we have:




```
    else
    {
        Node *current = *head;
        int i = 0;
        while (i < position - 1 && current != nullptr)
        {
            current = current->next;
            i++;
        }
```

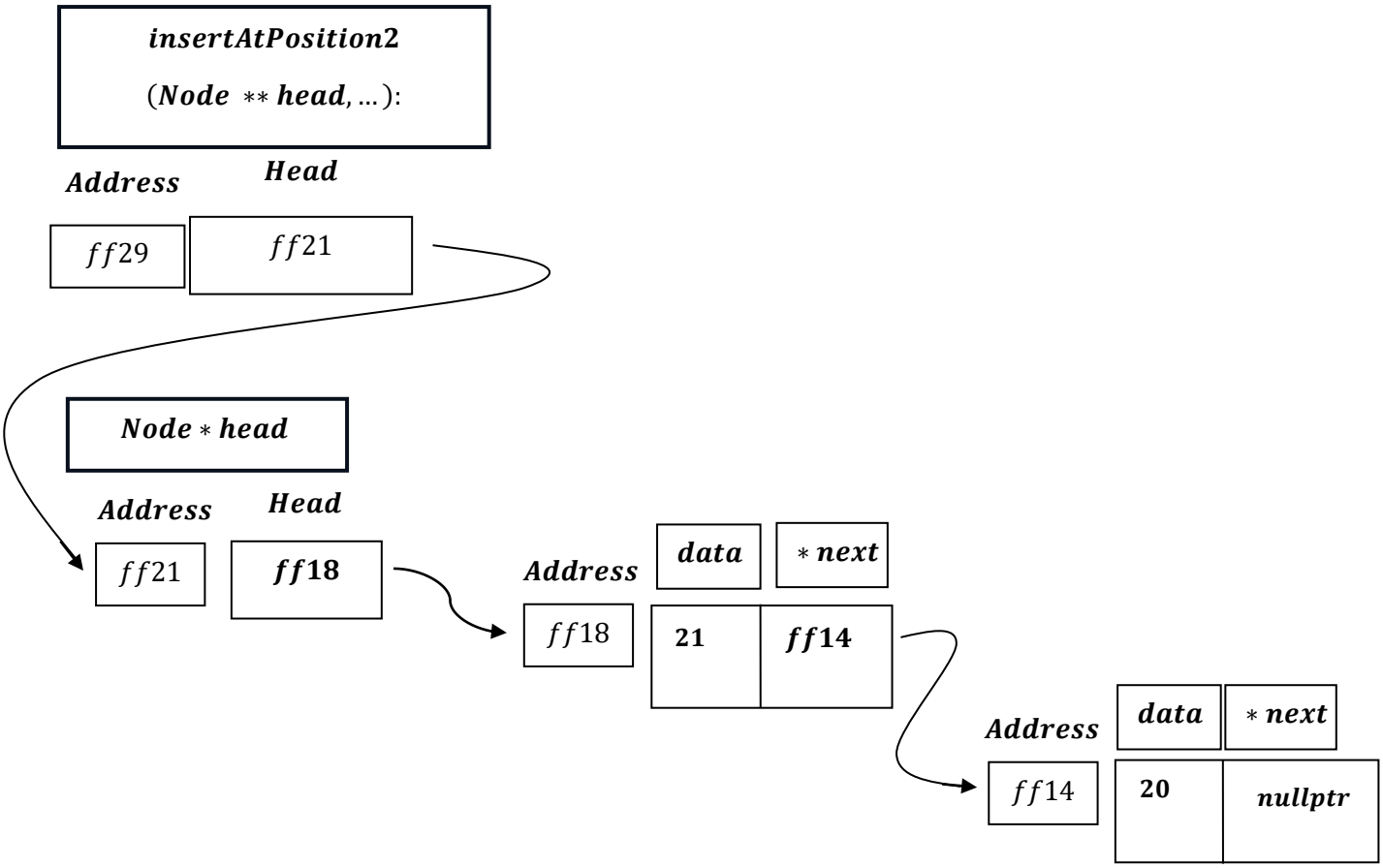
```
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }

        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
    }
```

$Now, say\ we\ pass\ data : 19\ at\ position : 2.$

```
    case 3:
        {
            int data,position;
            cout << "Enter the data: ";
            cin >> data;
            cout << "Enter the position: ";
            cin >> position;
            insertAtPosition(&head, data, position);
            break;
        }
```

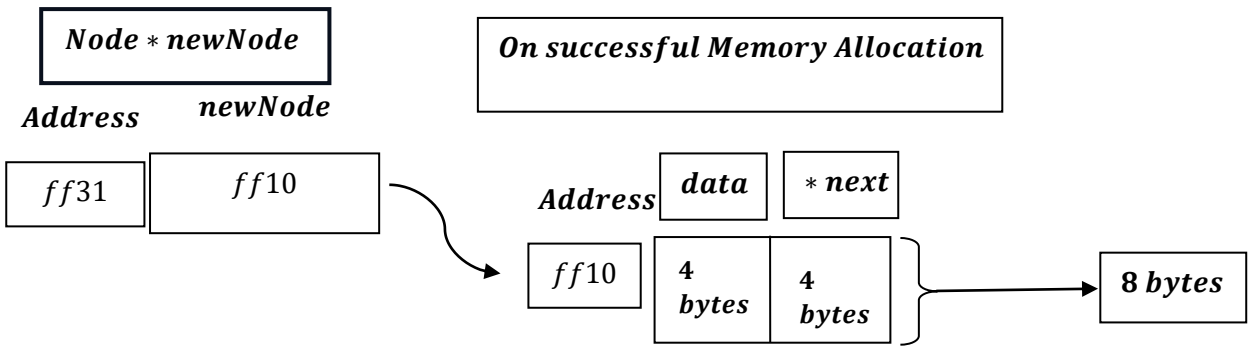
$InsertAtPosition2(ff21, 19, 2);$

```
void insertAtPosition2(Node **head, int data, int position) {…}
```



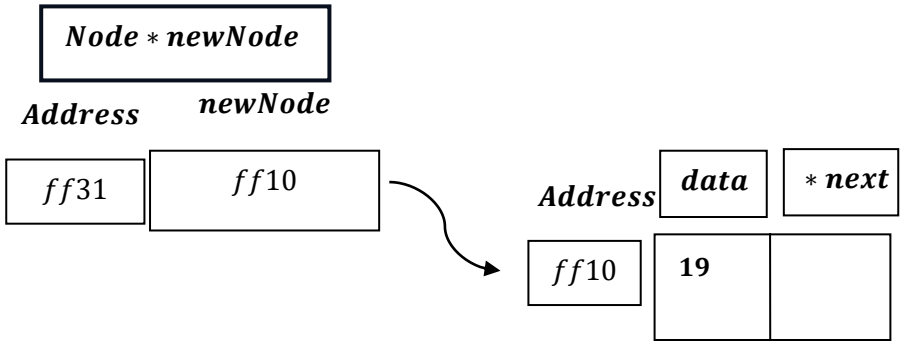
Next:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```



Going with successful memory allocation.

Next:

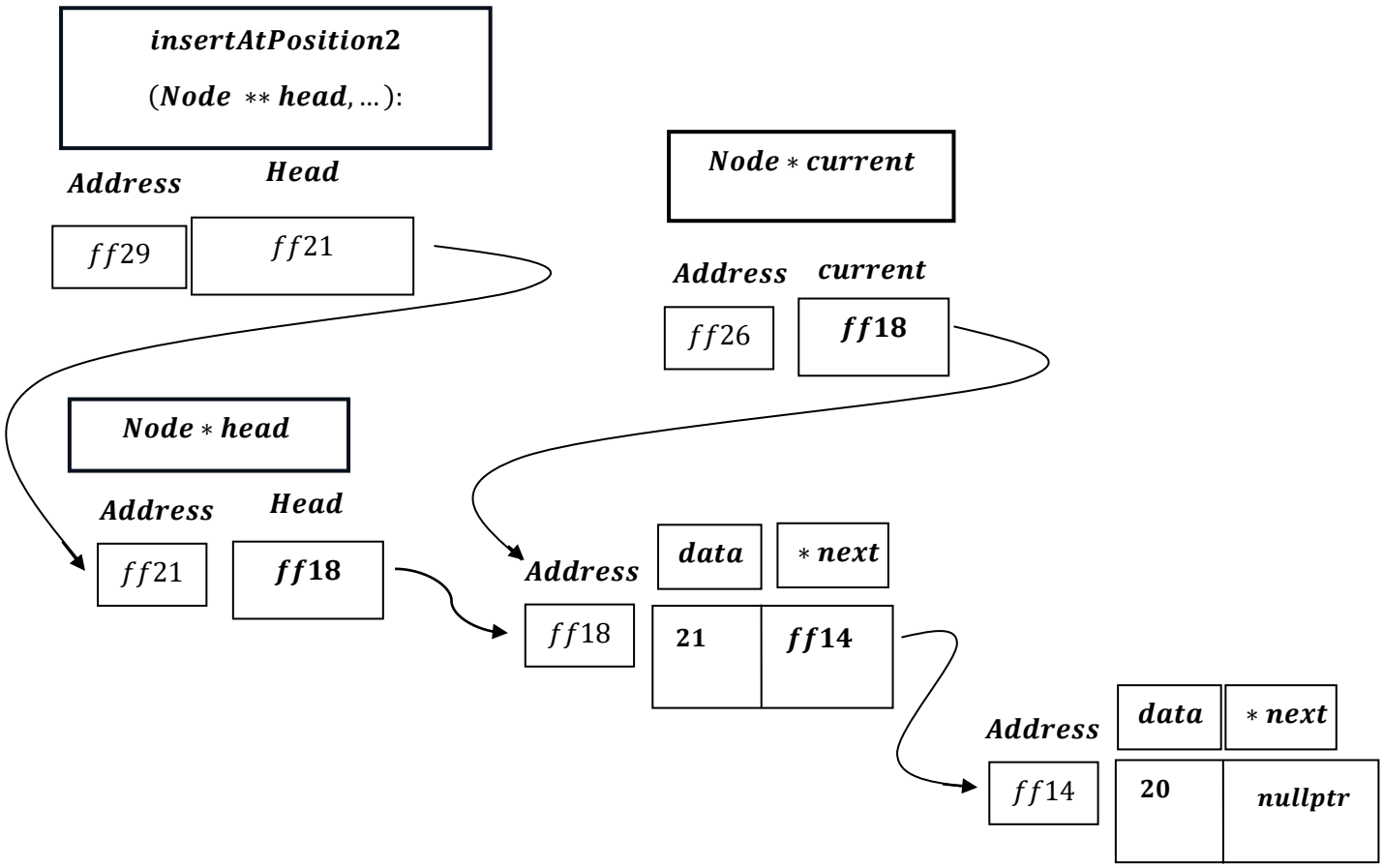


As: ***InsertAtPosition2***$(ff21, 19, 2)$;

Now, coming to else part:

```
        Node *current = *head;
```

current is a pointer of Node structure ,whose value is assigned with pointer head's value.



$Why\ ff18\ ?\ ,when\ we\ are\ using * head, i.e. here\ head\ used\ is\ of\ local\ pointer$
$to\ pointer\ i.e. Node ** head\ , and\ as * head\ is\ used, it\ will\ first\ derefer\ from$
$Node ** head\ and\ refer\ us\ to\ Node * head\ , which\ will\ cause\ current\ pointer\ to$
$store\ value\ contained\ in\ Node * head\ \ i.e. ff18\ , address\ of\ Head\ node\ to\ which$
$Node * head\ pointer\ points\ to.$

```
int i = 0;
 while (i < position - 1 && current != nullptr)
 {
     current = current->next;
     i++;
 }
```

$Our\ position\ is : 2\ , hence\ position - 1\ will\ become\ 2 - 1 = 1\ and$
$i < position - 1\ i.e. i < 1\ , hence\ i\ will\ looped\ for\ once\ only\ and\ at\ this$
$rate\ , current\ will\ never\ become\ nullptr\ as\ current \rightarrow next\ will\ not\ reach$
$the\ end\ node\ which\ contain\ next\ pointer\ value :\ \ nullptr\ , if$
$position\ is\ \ between\ 0\ to\ N - 1\ i.e. suppose\ length\ of\ linked\ list\ is\ 8\ , then$
$node\ can\ be\ inserted\ at\ position : 0, 1, 2, \ldots., 8\ \ and\ current\ will\ never$
$carry\ `nullptr`\ .$

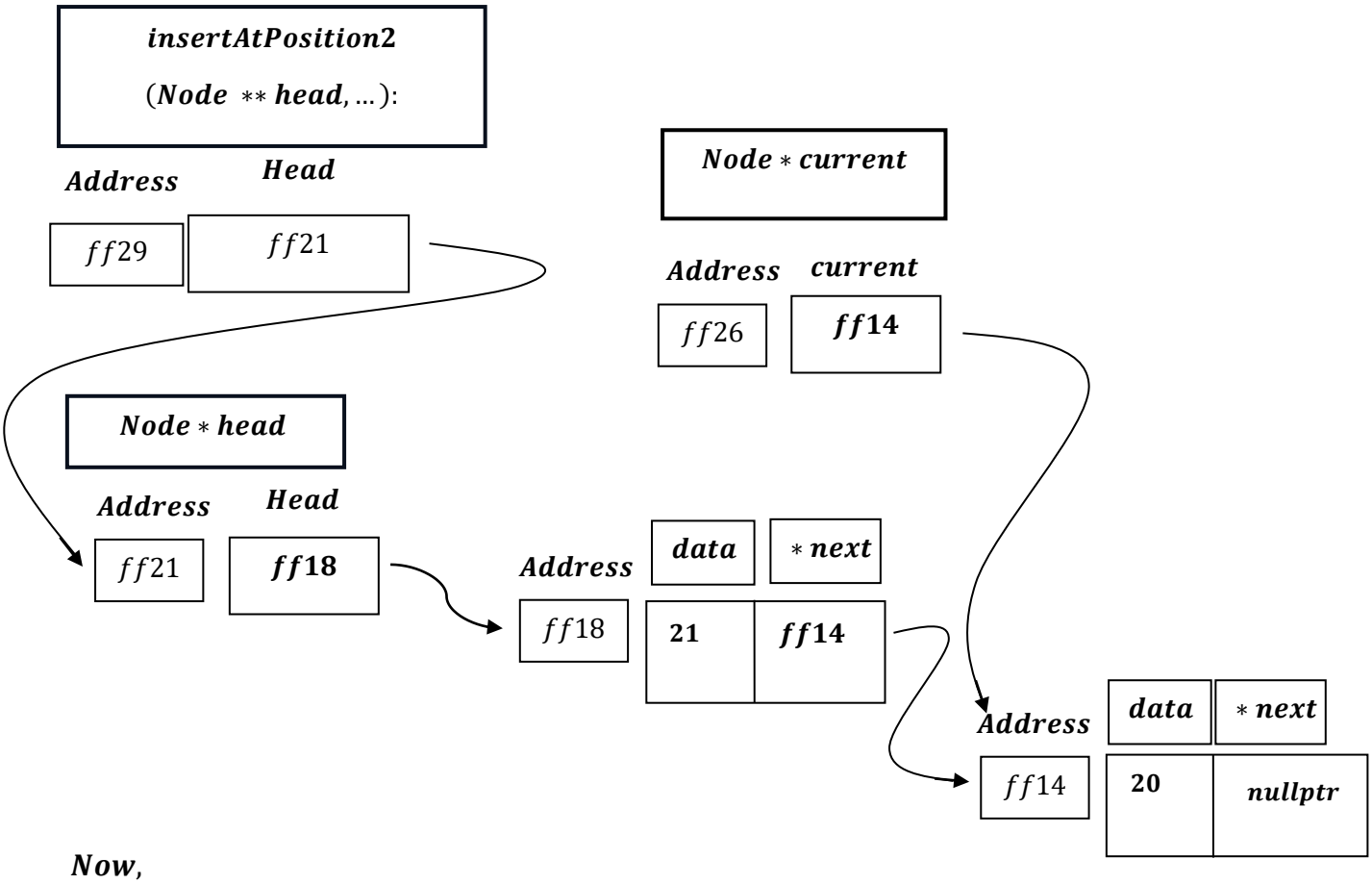
$But\ if\ position\ is\ 9\ it\ will\ return\ Nullptr, loop\ will\ get\ terminated\ .$

$Now\ our\ length\ of\ the\ linked\ list\ is : 2\ and\ we\ want\ to\ insert\ data\ at\ node$
$position : 2\ i.e. after\ 0\ and\ 1\ which\ is\ at\ n^{th}\ node.$

$when\ i = 0\ \ < (2 - 1) i.e. 1\ and\ current\ is\ not\ equal\ to\ `nullptr` :$

```
current = current->next;
```

$i.e. current\ pointer's\ value\ is\ assigned\ to\ the\ value\ contained\ in$
$current \rightarrow\ \ next.$



Now,

```
            i++;
```

$i.e. i = i + 1 = 0 + 1 = 1.$

And , while$(1 < 1\ \ [False]$ *AND current*$! =$ *nullptr* $[True])$ *is flase , hence loop terminates here.*

Now, if current's value would have been nullptr then:

```
        if (current == nullptr)
        {
            cout << "Error: Invalid position." << endl;
            return;
        }
```

This code is used to check if the pointer `current` is a null pointer (`nullptr`). If `current` is nullptr, it means that it does not point to any memory location(nor any arbritrary memory location)or any object rather to a special null value.

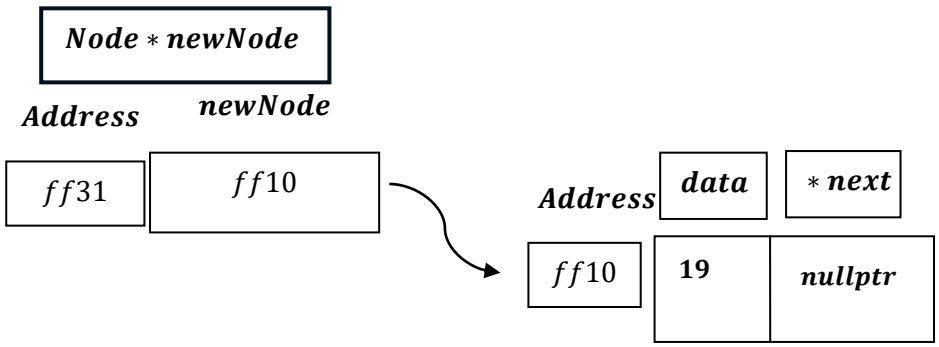
In this case, the code displays an error message `Error: Invalid position` *using the `cout` statement and then immediately `returns` from the InsertAtPosition2 method.*

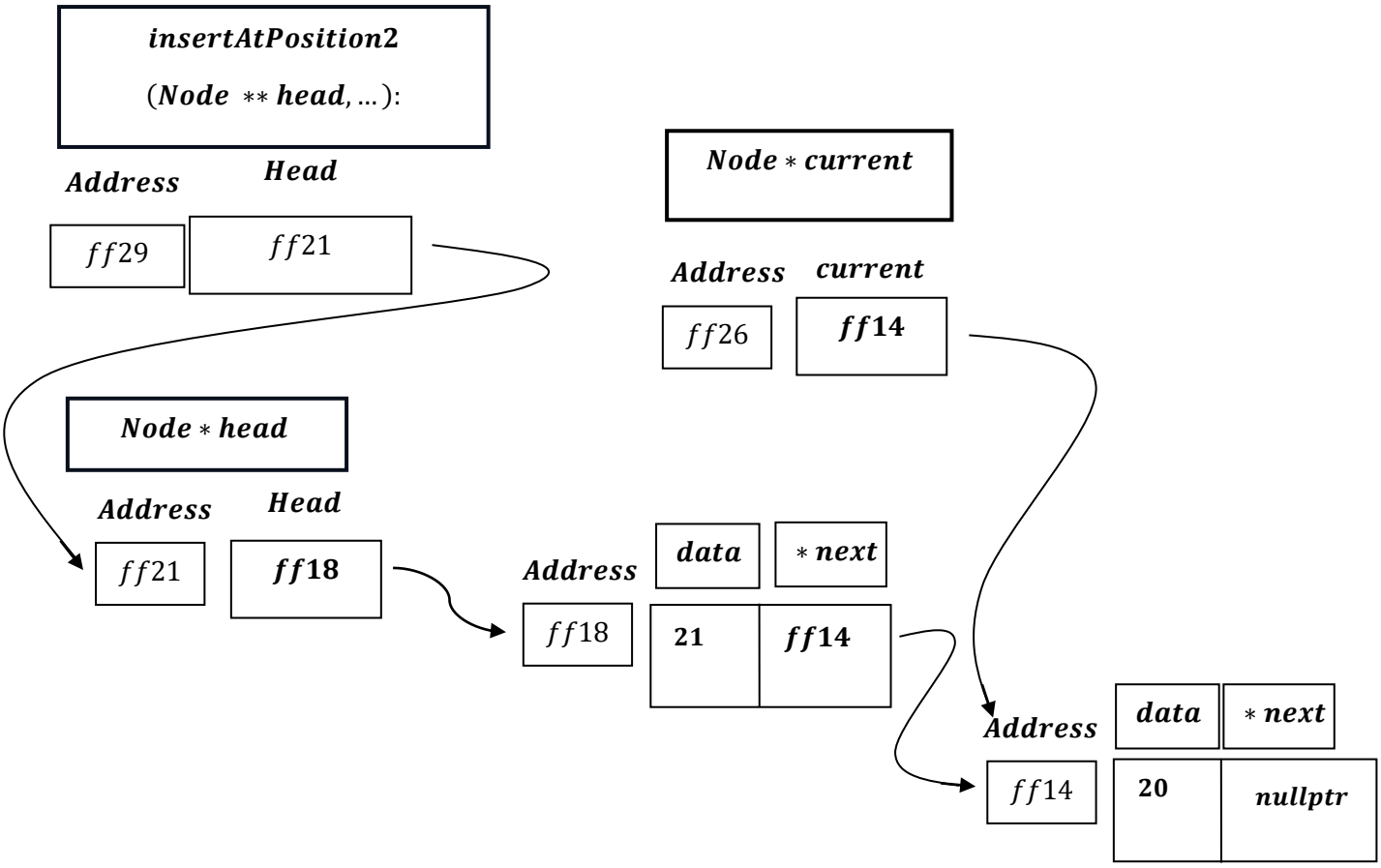
Next:

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
```

For:

```
            newNode->next = current->next;
```



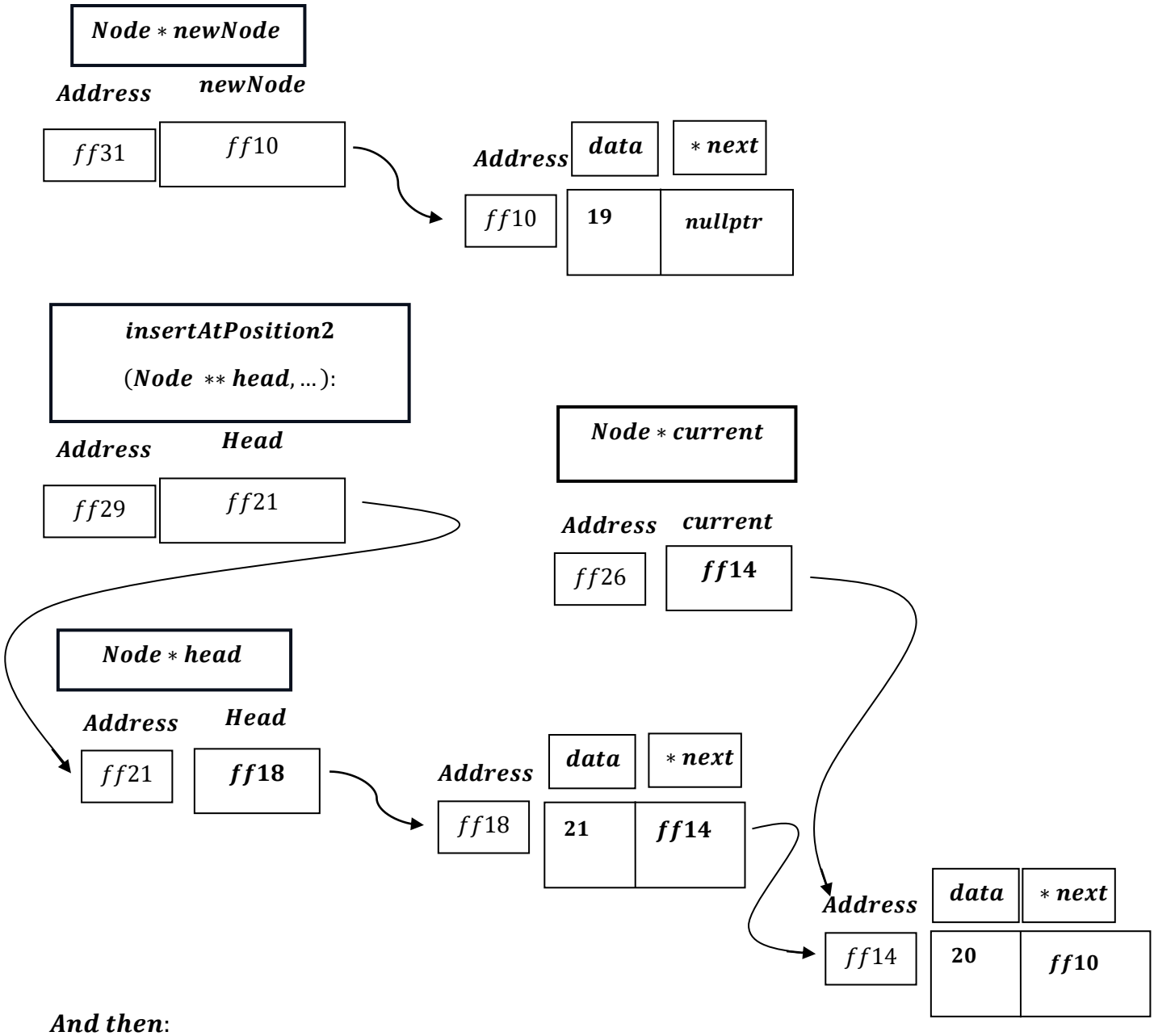


i.e. newNode → next's value is assigned to the value contained in current → next. And as per current scenario current pointer of Node's structure pointed to Adrress containing next pointer value as `nullptr` .

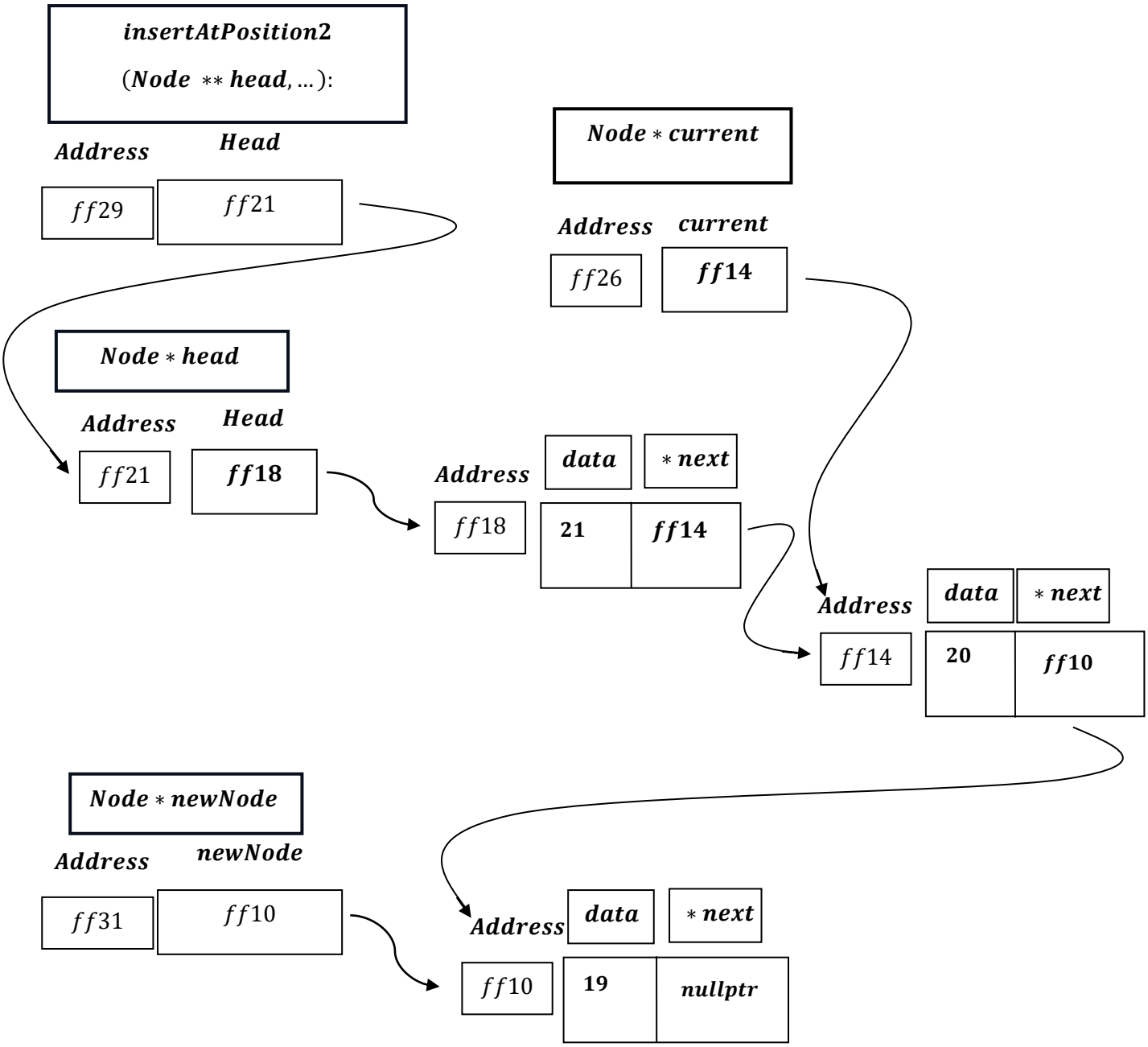
Next:

```
            current->next = newNode;
```

i.e. current → next's value is assigned to the value contained in newNode pointer.

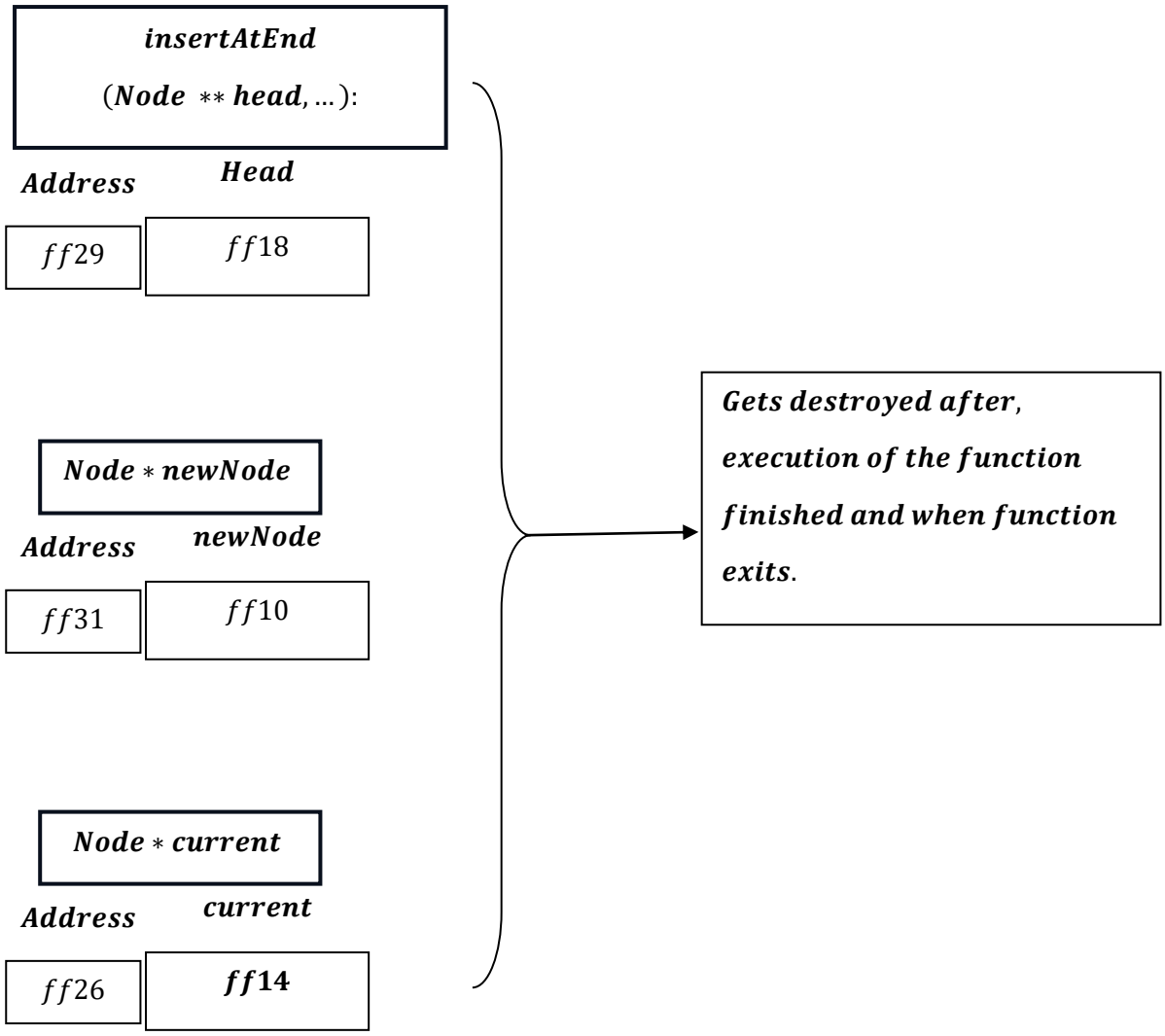

Node * newNode
Address
newNode
ff31
ff10
Address
data
* next
ff10
19
nullptr
insertAtPosition2
(Node ** head, ...):
Node * current
Address
Head
ff29
ff21
Address
current
ff26
ff14
Node * head
Address
Head
ff21
ff18
Address
data
* next
ff18
21
ff14
Address
data
* next
ff14
20
ff10

And then:

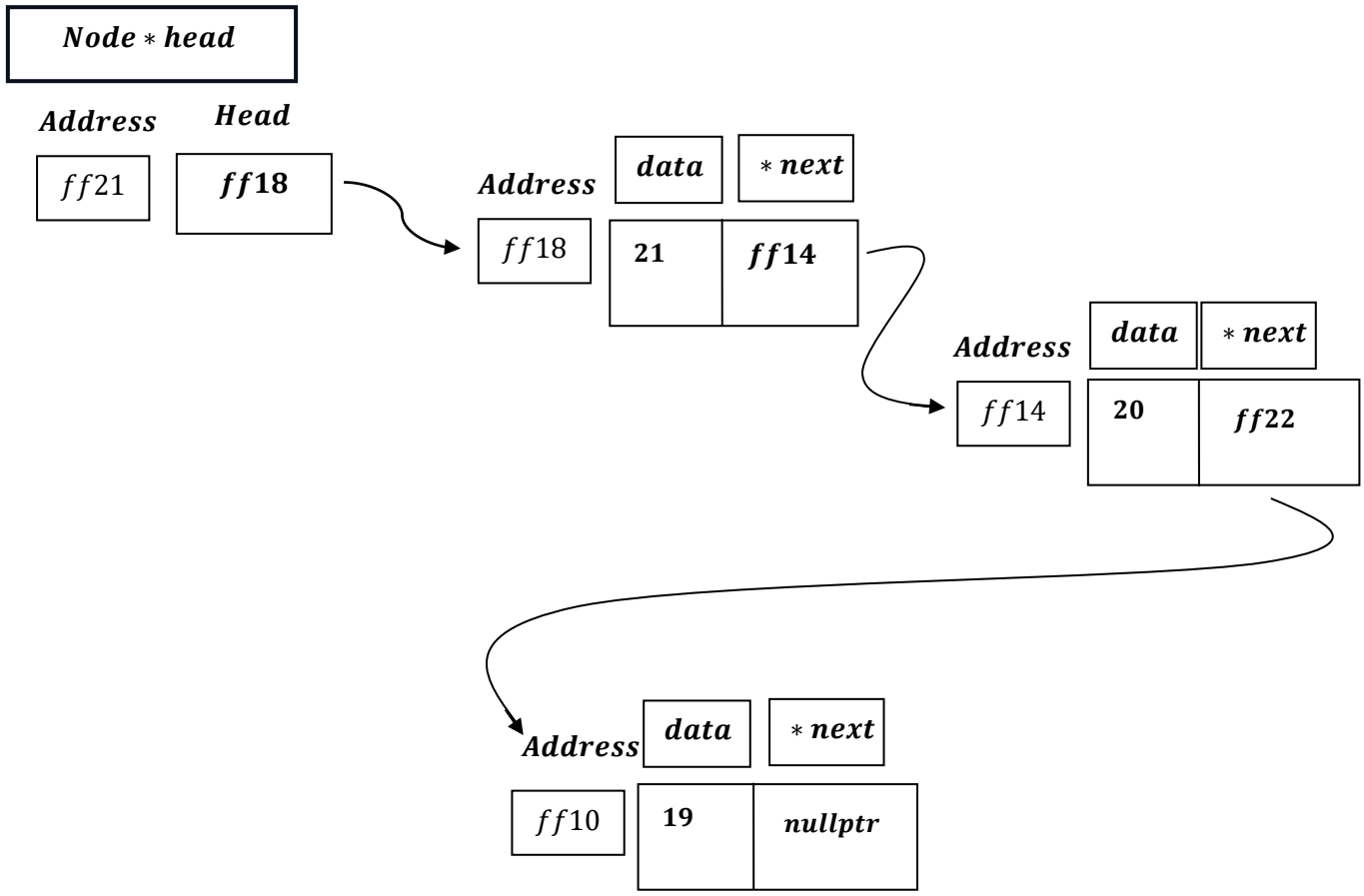


*Now, as Node * newNode , Node * current and Node ** head are temporary variable, therefore gets destroyed after function execution get finished .*

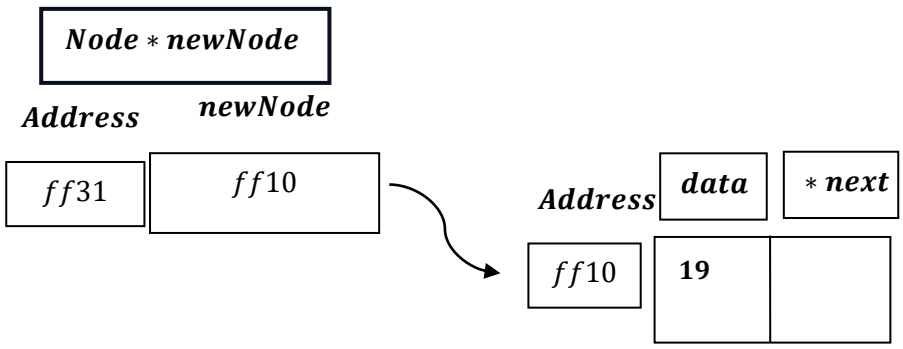
i.e.

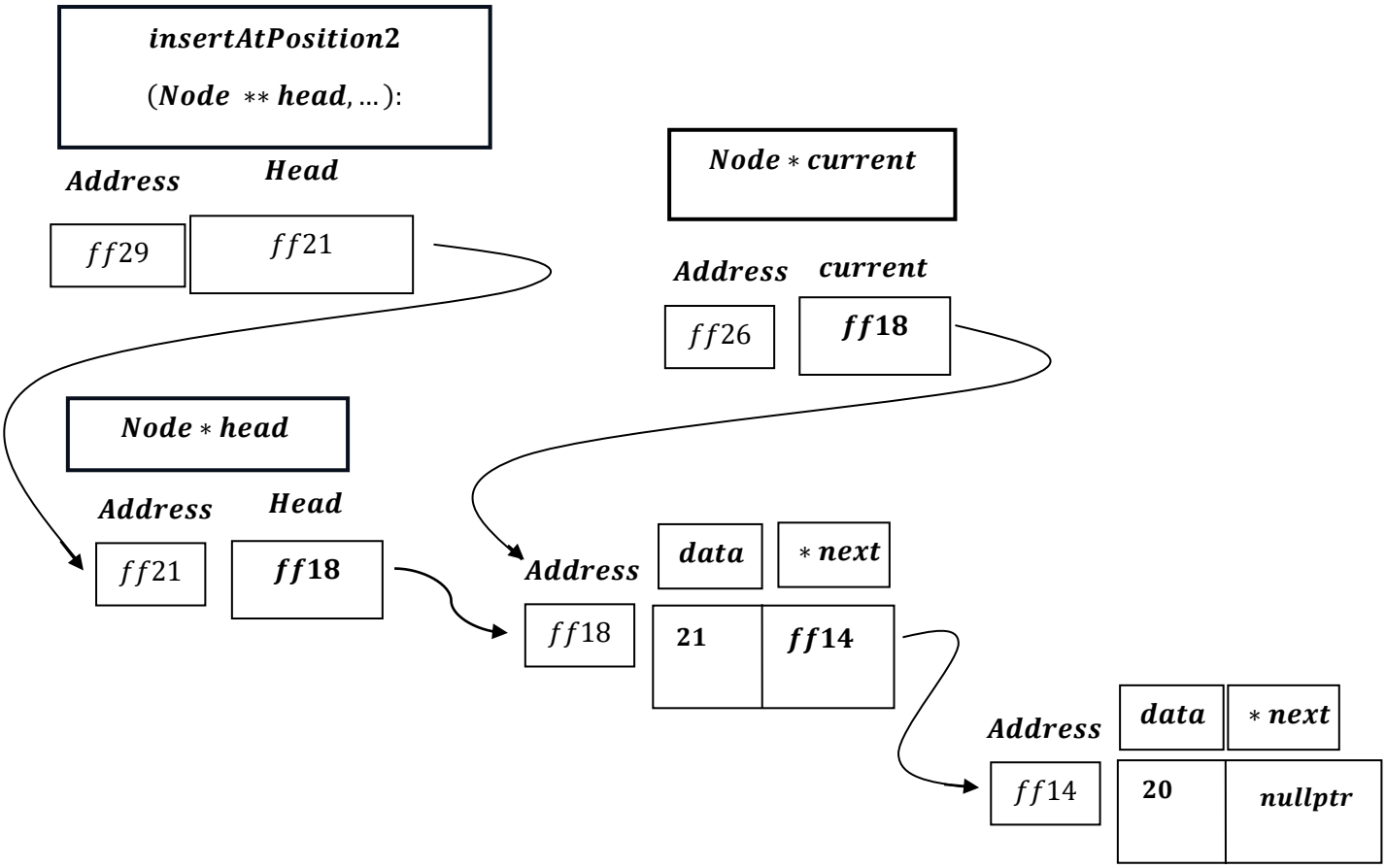


Hence we are left with the linked list:



Now , does loop runs when position is 1? When position = 1, we have already this scenario before loop execution starts:



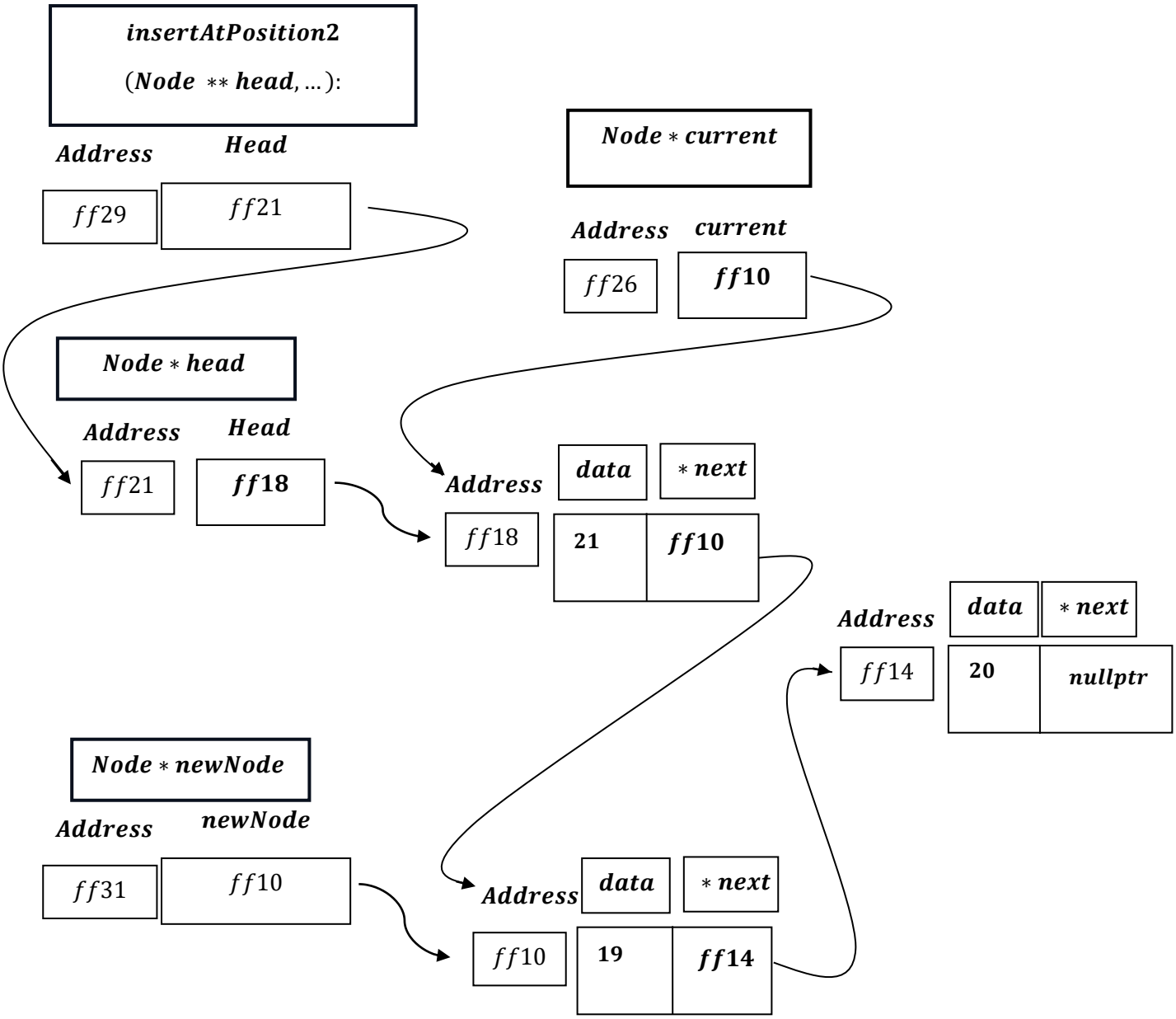


As current is pointing at Node 0 before Node 1 , we donot need further loop to get executed , also:

While $(i = 0 < (1 - 1) = 0$ [*is False*] ***and current*** $! =$ ***nullptr is true***$)$, ***the condition is false due to anding , hence directly we execute the else part:***

```
        else
        {
            newNode->next = current->next;
            current->next = newNode;
        }
```

Which does is:



Hence , no loop execution happens , when Position is 0 or Position is 1 .

The insertAtPosition2 function is designed to insert a new node with a given data value at a specified position in a singly linked list. Here is a step – by – step description of the process:

1. ***Memory Allocation for New Node:***

– The function starts by allocating memory for a new node using `malloc`.

$-$ *If the memory allocation fails (`newNode == nullptr`), it prints an error message and returns.*

2. *Assign Data to New Node*:

$-$ *The data value is assigned to the `data` field of the new node.*

3. *Insert at Head (Position 0)*:

$-$ *If the specified position is `0`, the new node is inserted at the head of the list.*

$-$ *The `next` pointer of the new node is set to the current head of the list.*

$-$ *The head pointer is updated to point to the new node.*

$-$ *The function then returns.*

4. *Check for Empty List*:

$-$ *If the list is empty (`* head == nullptr`), it prints an error message and returns.*

5. *Check for Invalid Position*:

$-$ *If the specified position is less than `0`, it prints an error message and returns.*

6. *Traverse to the Desired Position*:

$-$ *If the position is valid and the list is not empty, the function traverses the list to find the node after which the new node should be inserted.*

$-$ *A pointer `current` is initialized to point to the head of the list.*

$-$ *A counter `i` is initialized to `0`.*

$-$ *The function enters a loop that continues until `i` is less than `position − 1` and `current` is not `nullptr`.*

$-$ *Inside the loop, `current` is updated to point to the next node, and `i` is incremented.*

7. *Check for Invalid Position During Traversal*:

$-$ *If the traversal ends and `current` is `nullptr`, it means the specified position is beyond the end of the list.*

$-$ *In this case, it prints an error message and returns.*

8. *Insert the New Node*:

– *If the traversal is successful, the new node is inserted at the desired position.*

– *The `next` pointer of the new node is set to point to the node that `current–> next` was pointing to.*

– *The `next` pointer of `current` is updated to point to the new node.*

By following these steps, the function ensures that the new node is correctly inserted at the specified position in the linked list, handling various edge cases such as memory allocation failure, invalid positions, and empty lists.

Time Complexity of Insert At Position Function Module

```
void insertAtPosition(Node **head, int data, int position)
{
    Node *newNode = (Node *)malloc(sizeof(Node));   → O(1)

    if (newNode == nullptr)   → O(1)
    {
        cout << "Error: Memory allocation failed." << endl;   → O(1)
        return;   → O(1)
    }

    newNode->data = data;   → O(1)

    if (position == 0)   → O(1)
    {
        newNode->next = *head;   → O(1)
        *head = newNode;   → O(1)
        return;   → O(1)
    }

    if (*head == nullptr)   → O(1)
    {
        cout << "Error: List is empty." << endl;   → O(1)
        return;   → O(1)
    }

    if (position < 0)   → O(1)
    {
        cout << "Error: Invalid position." << endl;   → O(1)
        return;   → O(1)
    }
    else
    {
        Node *current = *head;   → O(1)

        int i = 0;   → O(1)
```

```
        while (i < position - 1 && current != nullptr)   → runs `n − 1` times at worst condition.
        {
            current = current->next;   → 1 + 1 + ⋯ to n − 1 times = O(n).
            i++;   → 1 + 1 + ⋯ to n − 1 times = O(n).
        }
        if (current == nullptr)   → O(1)
        {
            cout << "Error: Invalid position." << endl;   → O(1)
            return;   → O(1)
        }


        else
        {
            newNode->next = current->next;   → O(1)
            current->next = newNode;   → O(1)
        }
    }
}
```

There are three case scenarios:

→ 1. *Position is* 0 *and* 1 , *i.e. we want to insert data at first or second* . (*Best Case*)

→ 2. *Position is* N , *i.e. we want to insert data position at end* . (*Worst Case*)

→ 3. *Inserted position from* 2 *to* N, *while loop runs from* 1 *to* $N - 1$.

While if position is 0 *and* 1 , *it takes constant amount of time* : $O(1)$.

And when the position is uniformly distributed from 0 *to* N.

(*Average Case i.e. Undetermined Position*). *That is we donot whether the position will be* 0 *or* 1 *or* N, *where Node is to be inserted*.

$\underline{1.\ When\ Position\ is\ 0\ and\ 1(Best\ Case)}$

$\underline{When\ Position\ is\ 0}$:

$\rightarrow$ $The\ function\ begins\ by\ allocating\ memory\ for\ a\ new\ node\ using\ malloc.$
$This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $After\ allocating\ memory, the\ function\ checks\ if\ the\ allocation\ was$
$successful. If\ not, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ also\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ memory\ allocation\ is\ successful, the\ function\ initializes\ the\ new\ node's$
$data. This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ position\ is\ 0. If\ it\ is, the\ new\ node's\ next\ pointer$
$is\ set\ to\ the\ current\ head, and\ the\ head\ is\ updated\ to\ the\ new\ node.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ head\ pointer\ is\ pointed\ to$ `null pointer` $and\ if\ the$
$check\ is\ true\ , then\ outputs\ \ a\ message\ i.e.\ list\ is\ empty\ and\ this\ takes\ constant$
$time\ , O(1).$

$\rightarrow$ $if\ the\ \ position\ is\ less\ than\ 0, then\ it\ outputs\ message$ `Invalid Position`
$which\ takes\ constant\ time\ , O(1).$

$Total\ time\ complexity\ when\ Position\ is\ 0: -$

$$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1).$$

$We\ can\ say\ when\ we\ position\ is\ 0\ , we\ get\ the\ best\ time\ complexity:\ \Omega(1).$

When Position is 1:

→ *The function begins by allocating memory for a new node using malloc. This operation takes constant time, $O(1)$.*

→ *After allocating memory, the function checks if the allocation was successful. If not, it prints an error message and returns. This step also takes constant time, $O(1)$.*

→ *If memory allocation is successful, the function initializes the new node's data. This operation takes constant time, $O(1)$.*

→ *The function checks if the position is 0 and now the position is 1, hence if statement's condition checking will go false, the condition checking takes constant amount of time i.e. $O(1)$.*

→ *The function checks if the head pointer is pointed to `null pointer` and if the check is true , then outputs a message i.e. list is empty and this takes constant time , $O(1)$.*

→ *if the position is less than 0, then it outputs message `Invalid Position` which takes constant time , $O(1)$.*

→ *Now , position is nor 0 or head is neither `nullptr` , hence it goes to else part:*

→ *Now temporary current pointer of Node structure gets assigned to the value contained in head pointer which takes a constant amount of time i.e. $O(1)$.*

→ *Temporary variable `i` gets assigned to `0` which takes constant*

$amount\ of\ time\ i.e.\ O(1)$.

$\rightarrow$ $Now\ it\ initiates\ loop\ but\ it\ fails\ condition\ checking\ i.e.$
$while\ (i = 0 < (1 - 1) = 0)\ is\ false, which\ takes\ constant\ amount$
$of\ time\ i.e.\ O(1)$.

$\rightarrow$ $Next\ it\ moves\ on\ to\ check\ current\ pointers\ value\ is\ nullptr\ which\ is$
$not\ true\ here\ , this\ condition\ checking\ takes\ \ constant\ amount\ of\ time,$
$i.e.\ O(1)$.

$\rightarrow$ $As\ current\ is\ not\ nullptr\ , it\ moves\ to\ else\ part, there\ at\ first\ it\ assigns\ value$
$stored\ in\ next\ pointer\ of\ memory\ location\ pointed\ by\ newNode\ pointer\ of\ Node$
$structure\ to\ next\ pointer\ of\ memory\ location\ pointed\ by\ current\ pointer$
$of\ Node\ structure\ which\ takes\ constant\ amount\ of\ time\ i.e.\ O(1)$.

$\rightarrow$ $Secondly\ it\ assigns\ value\ stored\ in\ next\ pointer\ of\ memory\ location\ pointed$
$by\ current\ pointer\ of\ Node\ structure\ to\ newNode\ pointer\ of\ Node$
$structure\ which\ takes\ constant\ amount\ of\ time\ i.e.\ O(1)$.

$Total\ time\ complexity\ when\ Position\ is\ 1: -$

$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1) + O(1)$
$+O(1) = O(1)$.

$Or, We\ can\ say\ when\ we\ position\ is\ 1\ , we\ get\ the\ best\ time\ complexity:\ \Omega(1)$.

$\underline{2.\ When\ Position\ is\ N(Worst\ Case)}$

$\rightarrow$ $The\ function\ begins\ by\ allocating\ memory\ for\ a\ new\ node\ using\ malloc.$
$This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $After\ allocating\ memory, the\ function\ checks\ if\ the\ allocation\ was$
$successful. If\ not, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ also\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ memory\ allocation\ is\ successful, the\ function\ initializes\ the\ new\ node's$
$data. This\ operation\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ head\ of\ the\ list\ is\ null, indicating\ an\ empty\ list.$
$If\ the\ list\ is\ empty, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $The\ function\ checks\ if\ the\ position\ is\ less\ than\ 0.$
$If\ it\ is, it\ prints\ an\ error\ message\ and\ returns.$
$This\ step\ takes\ constant\ time, O(1).$

$\rightarrow$ $If\ the\ position\ is\ valid\ and\ the\ list\ is\ not\ empty, the\ function\ uses\ a$
$loop\ to\ traverse\ the\ list\ and\ find\ the\ node\ just\ before\ the\ desired$
$position. The\ number\ of\ iterations\ required\ is\ equal\ to\ position\ -\ 1.$
$In\ the\ worst\ case:$

$\rightarrow$ $If\ the\ position\ is\ 0, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow$ $If\ the\ position\ is\ 1, the\ loop's\ inner\ statement\ runs\ 0\ time.$
$\rightarrow$ $If\ the\ position\ is\ 2, the\ loop's\ inner\ statement\ runs\ 1\ times.$
$\rightarrow$ $If\ the\ position\ is\ 3, the\ loop's\ inner\ statement\ runs\ 2\ times.$

.

.

.

$\rightarrow$ $If\ the\ position\ is\ N, the\ loop's\ inner\ statement\ runs : N - 1\ times$,

$As\ when\ i = N - 1 < N - 1\ becomes\ false\ hence\ no\ execution\ of\ loop's\ inner\ statements.$

$Hence\ at\ worst\ case\ for\ position\ N\ \ loop\ run\ is\ \ N - 1, hence$

$for\ inner\ loop\ statements$:

$current\ =\ current \rightarrow next;$

$i++;$

$Also\ runs\ N - 1\ times\ each\ , hence, for\ two\ statements\ we\ can\ say, 2(N - 1)$

$and\ applying\ Big\ O\ i.e. O\big(2(N - 1)\big) = O(N - 1) = O(N)\ amount\ of\ time$

$approx$.

$\rightarrow$ $Once\ the\ loop\ reaches\ the\ desired\ position, the\ new\ node\ is\ inserted\ by$

$adjusting\ the\ next\ pointers. This\ operation\ takes\ constant\ time, O(1).$

$Overall\ Time\ Complexity$:

$O(1)\ +\ O(1)\ +\ O(1)\ +\ O(1)\ +\ O(1)\ +\ O(N)\ +\ O(1)\ =\ O(n)$

$Hence\ worst\ time\ complexity\ is : O(N).$

$$3.\ \textit{Inserted position from } 0 \textit{ to } N, \textit{while loop runs from } 0 \textit{ to } N-1.$$

$$\textit{And when the position is uniformly distributed from } 0 \textit{ to } N:$$

$$(\textit{Average Case i.e. Undetermined Position})$$

$\rightarrow$ *If the position is* 0, *the loop's inner statement runs* 0 *time.*

$\rightarrow$ *If the position is* 1, *the loop's inner statement runs* 0 *time.*

$\rightarrow$ *If the position is* 2, *the loop's inner statement runs* 1 *time.*

.

.

$\rightarrow$ *If the position is* (N), *the loop's inner statement runs* $(N-1)$ *times.*

As when $i = N-1 < N-1$ *becomes false, hence no execution of loop's inner statements.*

$$\textit{Hence}, \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2}.$$

Now, as we take all the possibilites then, we take the best case also i.e. $O(1)$, *when position* $= 1$ *and* 0. *Considering* `$T(P)$` *is time complexity of position* `$P$`.

$$T(P) = \begin{cases} 1 & , \textit{when } P = 0. \\ 1 & , \textit{when } P = 1. \\ P-1 & , \textit{when } P = 2, 3, 4, .., N. \end{cases}$$

Now, If there is `$N$` *no. of nodes there we can insert data at* $N+1$ *positions.*

$$\textit{For position } 0, \textit{the probability of insertion is } \frac{1}{N+1}.$$

$$\boldsymbol{For\ position\ 1, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}}.$$

.

.

$$\boldsymbol{For\ position\ N, the\ probability\ of\ insertion\ \ is \frac{1}{N+1}}.$$

$$\boldsymbol{Hence\ probability\ of\ insertion\ will\ always\ remain\ \frac{1}{N+1}}.$$

$\therefore$ $\boldsymbol{Average\ Time\ Complexity\ \ is:\ probability\ of\ insertion\ \times (number\ of\ possible\ iterations + constant\ time\ complexities\ for\ left\ over\ positions)}$.

$$= \left(\frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 2 + \cdots + \frac{1}{N+1} \times N - 1\right) + \frac{1}{N+1} \times 1 + \frac{1}{N+1} \times 1$$

$$= \frac{1}{N+1} \times \{(1 + 2 + 3 + \cdots + N - 1) + 1 + 1\}$$

$$= \frac{1}{N+1} \times \left\{\sum_{i=1}^{N-1} i + 2\right\}$$

$$= \frac{1}{N+1} \times \left(\frac{N(N-1)}{2} + 2\right)$$

$$= \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1}$$

$$= \frac{N^2 - N}{2N+2} + \frac{2}{N+1}$$

$\boldsymbol{Appplying\ Big\ Theta\ in\ the\ equation}$:

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

$$= \Theta\left(\frac{N^2 - N}{2N + 2}\right) + \Theta\left(\frac{2}{N + 1}\right)$$

***Now, taking out the dominant term of numerator and denominator*:**

$$= \Theta\left(\frac{N^2}{2N}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{2}{N}\right)$$

$$= \Theta\left(\frac{N}{2}\right) + 2 \times \Theta\left(\frac{1}{N}\right)$$

***Now removing the constant `2` we get*:**

$$= \Theta\left(\frac{N}{2}\right) + \Theta\left(\frac{1}{N}\right)$$

$$= \frac{1}{2} \times \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

***Now removing the constant `$\frac{1}{2}$` we get*:**

$$= \Theta(N) + \Theta\left(\frac{1}{N}\right)$$

***When combining two Big Theta* Θ terms , largest growth rate is the result:**

$\rightarrow \Theta(N)$ *grows linearly with* `N` .

$\rightarrow \Theta\left(\frac{1}{N}\right)$ *decreases to a constant , i.e. when N grows larger ,* $\frac{1}{N}$ *approaches to* 0.

This we can prove by limit and continuity also:

$$\lim_{n\to\infty}\left(\frac{1}{n}\right) \Rightarrow \frac{1}{\infty} = 0 \left[\frac{c}{\infty} = 0 \text{ , where c is constant}\right].$$

As $\Theta(N)'s$ *growth rate is higher , therefore , average case is* $\Theta(N)$.

Hence, Average Time Complexity is : $\Theta(N)$.

Note , by relation: $\Omega \leq \Theta \leq O$ *, we get*:

$$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N(N-1)}{2(N+1)} + \frac{2}{N+1} \leq c_2 \times N$$

Average Case Time Complexity[*Most Simplest Alternative Apporach*]:

(*Simplest approach*)

For position 0, *the probability of insertion is* $\frac{1}{N+1}$.

For position 1, *the probability of insertion is* $\frac{1}{N+1}$.

.

.

For position N, *the probability of insertion is* $\frac{1}{N+1}$.

$$\textit{Hence probability of insertion will always remain } \frac{1}{N+1}, \textit{for position } 0 \textit{ to } N.$$

$$\textit{Average case complexity}: \textit{ Probability of insertion } \times \begin{pmatrix} \textit{Summation} \\ \textit{of number of} \\ \textit{Position.} \end{pmatrix}$$

$$= \frac{1}{N+1} \times \left(\sum_{i=0}^{N} i \right)$$

$$= \frac{1}{N+1} \times \left(a_0 + \sum_{i=1}^{N} i \right) \left[\sum_{k=0}^{n} k = a_0 + \sum_{k=1}^{n} k \ [\textit{From series, rearranging sum}] \right]$$

$$= \frac{1}{N+1} \times \left(0 + \sum_{i=1}^{N} i \right) [\textit{We know}, a_0 = 0]$$

$$= \frac{1}{N+1} \times \left(\sum_{i=1}^{N} i \right)$$

$$= \frac{1}{N+1} \times \left(\frac{N(N+1)}{2} \right) \left[\sum_{i=1}^{N} i = \frac{N(N+1)}{2} \right]$$

$$= \frac{N}{2}$$

$$\textit{Hence, Average Time Complexity is}: \ \Theta\left(\frac{N}{2}\right) \textit{ or } \Theta(N).$$

$$\textit{Now,}$$

$$\textit{Note, by relation}: \Omega \leq \Theta \leq O, \textit{we get}:$$

$$\Longrightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$$

$$\Rightarrow c_1 \times 1 \leq \frac{N}{2} \leq c_2 \times N$$

Singly Linked List	***Cases***	***Time Complexity***
InsertAt Position2	**1.** ***Best Case***	$\Omega(1)$
	2. ***Worst Case***	$O(n)$
	3. ***Average Case***	$\Theta(n)$

****